

Heaps, Heapsort and Priority Queues

Dr. Bugra Caskurlu¹

¹ School of Computing
New Uzbekistan University

Overview

Sorting Algorithms

- **Insertion Sort:** *Incremental algorithm. Runs in $O(n^2)$ -time. Sorts in place.*

Overview

Sorting Algorithms

- **Insertion Sort:** *Incremental algorithm. Runs in $O(n^2)$ -time. Sorts in place.*
- **Merge Sort:** *Divide-and-conquer algorithm. Runs in $\Theta(n \cdot \lg n)$ -time. **Not** an in place algorithm, requires $\Theta(n)$ additional space.*

Overview

Sorting Algorithms

- **Insertion Sort:** *Incremental algorithm. Runs in $O(n^2)$ -time. Sorts in place.*
- **Merge Sort:** *Divide-and-conquer algorithm. Runs in $\Theta(n \cdot \lg n)$ -time. **Not** an in place algorithm, requires $\Theta(n)$ additional space.*
- **Effective use of data structures is important in designing efficient algorithms.** Let's bring the best of both world by using heaps.

Overview

Sorting Algorithms

- **Insertion Sort:** *Incremental algorithm. Runs in $O(n^2)$ -time. Sorts in place.*
- **Merge Sort:** *Divide-and-conquer algorithm. Runs in $\Theta(n \cdot \lg n)$ -time. **Not** an in place algorithm, requires $\Theta(n)$ additional space.*
- **Effective use of data structures is important in designing efficient algorithms.** Let's bring the best of both world by using heaps.
- **Heapsort:** *Uses heaps. Runs in $O(n \lg n)$ -time, and an in place algorithm.*

Overview

Sorting Algorithms

- **Insertion Sort:** *Incremental algorithm. Runs in $O(n^2)$ -time. Sorts in place.*
- **Merge Sort:** *Divide-and-conquer algorithm. Runs in $\Theta(n \cdot \lg n)$ -time. **Not** an in place algorithm, requires $\Theta(n)$ additional space.*
- **Effective use of data structures is important in designing efficient algorithms.** Let's bring the best of both world by using heaps.
- **Heapsort:** *Uses heaps. Runs in $O(n \lg n)$ -time, and an in place algorithm.*
- Is it possible to improve the running-time further?

Overview

Sorting Algorithms

- **Insertion Sort:** *Incremental algorithm. Runs in $O(n^2)$ -time. Sorts in place.*
- **Merge Sort:** *Divide-and-conquer algorithm. Runs in $\Theta(n \cdot \lg n)$ -time. **Not** an in place algorithm, requires $\Theta(n)$ additional space.*
- **Effective use of data structures is important in designing efficient algorithms.** Let's bring the best of both world by using heaps.
- **Heapsort:** *Uses heaps. Runs in $O(n \lg n)$ -time, and an in place algorithm.*
- Is it possible to improve the running-time further? Not quite, unless we have some assumptions on the input.

Overview

Sorting Algorithms

- **Insertion Sort:** *Incremental algorithm. Runs in $O(n^2)$ -time. Sorts in place.*
- **Merge Sort:** *Divide-and-conquer algorithm. Runs in $\Theta(n \cdot \lg n)$ -time. **Not** an in place algorithm, requires $\Theta(n)$ additional space.*
- **Effective use of data structures is important in designing efficient algorithms.** Let's bring the best of both world by using heaps.
- **Heapsort:** *Uses heaps. Runs in $O(n \lg n)$ -time, and an in place algorithm.*
- Is it possible to improve the running-time further? Not quite, unless we have some assumptions on the input. Every **comparison based sorting algorithm will take $\Omega(n \cdot \lg n)$ -time in the worst case.**

Definition

Example Heap

- The **(binary)** heap data structure is an array object that can be viewed as a **nearly** complete binary tree satisfying **heap property**.

Definition

Example Heap

- The **(binary)** heap data structure is an array object that can be viewed as a **nearly** complete binary tree satisfying **heap property**.
- Logical representation of a heap is a binary tree that is completely filled on all levels except possibly the lowest, which is filled from left up to a point.

Definition

Example Heap

- The **(binary)** heap data structure is an array object that can be viewed as a **nearly** complete binary tree satisfying **heap property**.
- Logical representation of a heap is a binary tree that is completely filled on all levels except possibly the lowest, which is filled from left up to a point.
- **Ex:** Logical representation of the array object [16, 14, 10, 8, 7, 9, 3, 2, 4, 1] is a tree with 10 nodes. At level 0, it has 1 element (16). At level 1, it has two elements (14 and 10). At level 2, it has four elements (8, 7, 9 and 3). At level 3, it has the remaining three elements (2, 4, and 1) filled from left to right.

Definition

Example Heap

- The **(binary)** heap data structure is an array object that can be viewed as a **nearly** complete binary tree satisfying **heap property**.
- Logical representation of a heap is a binary tree that is completely filled on all levels except possibly the lowest, which is filled from left up to a point.
- **Ex:** Logical representation of the array object [16, 14, 10, 8, 7, 9, 3, 2, 4, 1] is a tree with 10 nodes. At level 0, it has 1 element (16). At level 1, it has two elements (14 and 10). At level 2, it has four elements (8, 7, 9 and 3). At level 3, it has the remaining three elements (2, 4, and 1) filled from left to right.
- In this example, all the elements of the array are part of the heap, but in general, this may not be the case. Thus, an array *A* that represents a heap is an object with the following two attributes.
- **length[A]**: Number of elements in the array.
- **heapsize[A]**: Number of elements in the heap stored within array *A*.

Definition

Example Heap

- The **(binary)** heap data structure is an array object that can be viewed as a **nearly** complete binary tree satisfying **heap property**.
- Logical representation of a heap is a binary tree that is completely filled on all levels except possibly the lowest, which is filled from left up to a point.
- **Ex:** Logical representation of the array object [16, 14, 10, 8, 7, 9, 3, 2, 4, 1] is a tree with 10 nodes. At level 0, it has 1 element (16). At level 1, it has two elements (14 and 10). At level 2, it has four elements (8, 7, 9 and 3). At level 3, it has the remaining three elements (2, 4, and 1) filled from left to right.
- In this example, all the elements of the array are part of the heap, but in general, this may not be the case. Thus, an array *A* that represents a heap is an object with the following two attributes.
- **length[A]**: Number of elements in the array.
- **heapsize[A]**: Number of elements in the heap stored within array *A*.
- Notice that **heapsize[A] ≤ length[A]**.

Properties

Root, Parent, and Children

- **Root:** The root of the tree is $A[1]$.

Properties

Root, Parent, and Children

- **Root:** The root of the tree is $A[1]$.
- **Parent and Children:** Given the index i of a node the indices of its parent and children can be computed by the following macros (or inline procedures):
- **PARENT**(i) return $\lfloor \frac{i}{2} \rfloor$;
- **LEFT**(i) return $2 \cdot i$;
- **RIGHT**(i) return $2 \cdot i + 1$;

Properties

Root, Parent, and Children

- **Root:** The root of the tree is $A[1]$.
- **Parent and Children:** Given the index i of a node the indices of its parent and children can be computed by the following macros (or inline procedures):
- **PARENT**(i) return $\lfloor \frac{i}{2} \rfloor$;
- **LEFT**(i) return $2 \cdot i$;
- **RIGHT**(i) return $2 \cdot i + 1$;

Heap Property

- **Max-Heap Property:** $A[\text{PARENT}(i)] \geq A[i]$ for all $i > 1$.
- **Min-Heap Property:** $A[\text{PARENT}(i)] \leq A[i]$ for all $i > 1$.

Properties

Root, Parent, and Children

- **Root:** The root of the tree is $A[1]$.
- **Parent and Children:** Given the index i of a node the indices of its parent and children can be computed by the following macros (or inline procedures):
- **PARENT**(i) return $\lfloor \frac{i}{2} \rfloor$;
- **LEFT**(i) return $2 \cdot i$;
- **RIGHT**(i) return $2 \cdot i + 1$;

Heap Property

- **Max-Heap Property:** $A[\text{PARENT}(i)] \geq A[i]$ for all $i > 1$.
- **Min-Heap Property:** $A[\text{PARENT}(i)] \leq A[i]$ for all $i > 1$.

Proposition

In a max-heap, the largest element is stored at the root. In a min-heap, the smallest element is stored at the root.

Definitions and Exercises

Definition

The **height** of a node in a heap is defined as the number of edges on the longest simple downward path from the node to a leaf. The height of the heap is the height of the root.

Definitions and Exercises

Definition

The **height** of a node in a heap is defined as the number of edges on the longest simple downward path from the node to a leaf. The height of the heap is the height of the root.

Exercises

- What is the minimum and maximum numbers of elements in a heap of height h ?

Definitions and Exercises

Definition

The **height** of a node in a heap is defined as the number of edges on the longest simple downward path from the node to a leaf. The height of the heap is the height of the root.

Exercises

- What is the minimum and maximum numbers of elements in a heap of height h ? Min: 2^h .
Max: $2^{(h+1)} - 1$.

Definitions and Exercises

Definition

The **height** of a node in a heap is defined as the number of edges on the longest simple downward path from the node to a leaf. The height of the heap is the height of the root.

Exercises

- What is the minimum and maximum numbers of elements in a heap of height h ? Min: 2^h . Max: $2^{(h+1)} - 1$.
- Show that an n -element heap has height $\lfloor \lg n \rfloor$.

Definitions and Exercises

Definition

The **height** of a node in a heap is defined as the number of edges on the longest simple downward path from the node to a leaf. The height of the heap is the height of the root.

Exercises

- What is the minimum and maximum numbers of elements in a heap of height h ? Min: 2^h . Max: $2^{(h+1)} - 1$.
- Show that an n -element heap has height $\lfloor \lg n \rfloor$.
- **Proof:** Let h be the height of the heap. Then, $2^h \leq n < 2^{(h+1)}$. Taking logs gives us $h \leq \lg n < (h+1)$. Since h is an integer, we have $h = \lfloor \lg n \rfloor$.

MAX-HEAPIFY

Pseudocode

```
MAX-HEAPIFY(A, i)
    l = LEFT(i);
    r = RIGHT(i);

    if l ≤ heap-size[A] and A[l] > A[i]
        largest = l;
    else
        largest = i;

    if r ≤ heap-size[A] and A[r] > A[largest]
        largest = r;

    if largest ≠ i
        exchange A[i] ↔ A[largest];
        MAX-HEAPIFY(A, largest);
```

Explanation and Analysis

Explanation

- **Inputs:** an array A , and an index i .

Explanation and Analysis

Explanation

- **Inputs:** an array A , and an index i .
- **Assumption on the input:** When **MAX-HEAPIFY** is called it is assumed that the binary trees rooted at **LEFT**(i) and **RIGHT**(i) are max-heaps, but that $A[i]$ may be smaller than its children, thus violating the heap-property.

Explanation and Analysis

Explanation

- **Inputs:** an array A , and an index i .
- **Assumption on the input:** When **MAX-HEAPIFY** is called it is assumed that the binary trees rooted at **LEFT**(i) and **RIGHT**(i) are max-heaps, but that $A[i]$ may be smaller than its children, thus violating the heap-property.
- **Output:** The function **MAX-HEAPIFY** lets the value of $A[i]$ float down in the max-heap so that the subtree rooted at index i becomes a max-heap.

Explanation and Analysis

Explanation

- **Inputs:** an array A , and an index i .
- **Assumption on the input:** When **MAX-HEAPIFY** is called it is assumed that the binary trees rooted at **LEFT**(i) and **RIGHT**(i) are max-heaps, but that $A[i]$ may be smaller than its children, thus violating the heap-property.
- **Output:** The function **MAX-HEAPIFY** lets the value of $A[i]$ float down in the max-heap so that the subtree rooted at index i becomes a max-heap.

Analysis

- **Analysis 1:** **MAX-HEAPIFY** takes $\Theta(1)$ time plus the recursive call on a node whose height is one less. Since the height of a heap is at most $\lg n$, **MAX-HEAPIFY** takes $O(\lg n)$ -time.

Explanation and Analysis

Explanation

- **Inputs:** an array A , and an index i .
- **Assumption on the input:** When **MAX-HEAPIFY** is called it is assumed that the binary trees rooted at **LEFT**(i) and **RIGHT**(i) are max-heaps, but that $A[i]$ may be smaller than its children, thus violating the heap-property.
- **Output:** The function **MAX-HEAPIFY** lets the value of $A[i]$ float down in the max-heap so that the subtree rooted at index i becomes a max-heap.

Analysis

- **Analysis 1:** **MAX-HEAPIFY** takes $\Theta(1)$ time plus the recursive call on a node whose height is one less. Since the height of a heap is at most $\lg n$, **MAX-HEAPIFY** takes $O(\lg n)$ -time. **Its running-time is not $\Theta(\lg n)$** , since its best case performance is $\Theta(1)$.

Explanation and Analysis

Explanation

- **Inputs:** an array A , and an index i .
- **Assumption on the input:** When **MAX-HEAPIFY** is called it is assumed that the binary trees rooted at **LEFT**(i) and **RIGHT**(i) are max-heaps, but that $A[i]$ may be smaller than its children, thus violating the heap-property.
- **Output:** The function **MAX-HEAPIFY** lets the value of $A[i]$ float down in the max-heap so that the subtree rooted at index i becomes a max-heap.

Analysis

- **Analysis 1:** **MAX-HEAPIFY** takes $\Theta(1)$ time plus the recursive call on a node whose height is one less. Since the height of a heap is at most $\lg n$, **MAX-HEAPIFY** takes $O(\lg n)$ -time. **Its running-time is not $\Theta(\lg n)$** , since its best case performance is $\Theta(1)$.
- **Analysis 2:** Worst-case performance of **MAX-HEAPIFY** is can be expressed as a recurrence since it is a recursive algorithm.

Explanation and Analysis

Explanation

- **Inputs:** an array A , and an index i .
- **Assumption on the input:** When **MAX-HEAPIFY** is called it is assumed that the binary trees rooted at **LEFT**(i) and **RIGHT**(i) are max-heaps, but that $A[i]$ may be smaller than its children, thus violating the heap-property.
- **Output:** The function **MAX-HEAPIFY** lets the value of $A[i]$ float down in the max-heap so that the subtree rooted at index i becomes a max-heap.

Analysis

- **Analysis 1:** **MAX-HEAPIFY** takes $\Theta(1)$ time plus the recursive call on a node whose height is one less. Since the height of a heap is at most $\lg n$, **MAX-HEAPIFY** takes $O(\lg n)$ -time. **Its running-time is not $\Theta(\lg n)$** , since its best case performance is $\Theta(1)$.
- **Analysis 2:** Worst-case performance of **MAX-HEAPIFY** is can be expressed as a recurrence since it is a recursive algorithm. The children's subtrees have size at most $\frac{2 \cdot n}{3}$, and thus the worst-case running time can be expressed by the recurrence $T(n) = T(\frac{2 \cdot n}{3}) + \Theta(1)$. The solution to this recurrence is $\Theta(\lg n)$.

Explanation and Analysis

Explanation

- **Inputs:** an array A , and an index i .
- **Assumption on the input:** When **MAX-HEAPIFY** is called it is assumed that the binary trees rooted at **LEFT**(i) and **RIGHT**(i) are max-heaps, but that $A[i]$ may be smaller than its children, thus violating the heap-property.
- **Output:** The function **MAX-HEAPIFY** lets the value of $A[i]$ float down in the max-heap so that the subtree rooted at index i becomes a max-heap.

Analysis

- **Analysis 1:** **MAX-HEAPIFY** takes $\Theta(1)$ time plus the recursive call on a node whose height is one less. Since the height of a heap is at most $\lg n$, **MAX-HEAPIFY** takes $O(\lg n)$ -time. **Its running-time is not $\Theta(\lg n)$** , since its best case performance is $\Theta(1)$.
- **Analysis 2:** Worst-case performance of **MAX-HEAPIFY** is can be expressed as a recurrence since it is a recursive algorithm. The children's subtrees have size at most $\frac{2 \cdot n}{3}$, and thus the worst-case running time can be expressed by the recurrence $T(n) = T(\frac{2 \cdot n}{3}) + \Theta(1)$. The solution to this recurrence is $\Theta(\lg n)$.
- Wait a minute! Didn't we just say that the running time is not $\Theta(\lg n)$.

Explanation and Analysis

Explanation

- **Inputs:** an array A , and an index i .
- **Assumption on the input:** When **MAX-HEAPIFY** is called it is assumed that the binary trees rooted at **LEFT**(i) and **RIGHT**(i) are max-heaps, but that $A[i]$ may be smaller than its children, thus violating the heap-property.
- **Output:** The function **MAX-HEAPIFY** lets the value of $A[i]$ float down in the max-heap so that the subtree rooted at index i becomes a max-heap.

Analysis

- **Analysis 1:** **MAX-HEAPIFY** takes $\Theta(1)$ time plus the recursive call on a node whose height is one less. Since the height of a heap is at most $\lg n$, **MAX-HEAPIFY** takes $O(\lg n)$ -time. **Its running-time is not $\Theta(\lg n)$** , since its best case performance is $\Theta(1)$.
- **Analysis 2:** Worst-case performance of **MAX-HEAPIFY** is can be expressed as a recurrence since it is a recursive algorithm. The children's subtrees have size at most $\frac{2 \cdot n}{3}$, and thus the worst-case running time can be expressed by the recurrence $T(n) = T(\frac{2 \cdot n}{3}) + \Theta(1)$. The solution to this recurrence is $\Theta(\lg n)$.
- Wait a minute! Didn't we just say that the running time is not $\Theta(\lg n)$. The recurrence is for the worst-case performance. It is only $\Theta(\lg n)$ in the worst case, such as **INSERTION-SORT** is $\Theta(n^2)$ in the worst case.

Building Max-Heap

Strategy

Strategy: We can use the procedure **MAX-HEAPIFY** in a bottom-up manner for all **nonleaf nodes**, to convert an array $A = [1 \dots n]$ into a max-heap. So, we will have an *incremental algorithm*.

Building Max-Heap

Strategy

Strategy: We can use the procedure **MAX-HEAPIFY** in a bottom-up manner for all **nonleaf nodes**, to convert an array $A = [1 \dots n]$ into a max-heap. So, we will have an *incremental algorithm*.

Pseudocode

```
BUILD-MAX-HEAP (A)
    heap-size[A] = length[A];

    for  $i = \lfloor \text{length}[A]/2 \rfloor$  downto 1
        MAX-HEAPIFY (A, i);
```

Building Max-Heap

Strategy

Strategy: We can use the procedure **MAX-HEAPIFY** in a bottom-up manner for all **nonleaf nodes**, to convert an array $A = [1 \dots n]$ into a max-heap. So, we will have an *incremental algorithm*.

Pseudocode

```
BUILD-MAX-HEAP ( $\bar{A}$ )  
    heap-size[ $\bar{A}$ ] = length[ $\bar{A}$ ];  
  
    for  $i = \lfloor \text{length}[\bar{A}] / 2 \rfloor$  downto 1  
        MAX-HEAPIFY ( $\bar{A}$ ,  $i$ );
```

Loop Invariant

At the start of each iteration of the **for** loop, each node $(i+1), (i+2), \dots, n$ is the root of a max-heap.

Analysis of Buid-Max-Heap

EE Analysis

Each call to **MAX-HEAPIFY** costs $O(\lg n)$, and there are $O(n)$ such calls. Thus, the running time is $O(n \cdot \lg n)$.

Analysis of Buid-Max-Heap

EE Analysis

Each call to **MAX-HEAPIFY** costs $O(\lg n)$, and there are $O(n)$ such calls. Thus, the running time is $O(n \cdot \lg n)$.

CS Analysis

- The time for **MAX-HEAPIFY** to run at a node varies with the height of the node in the heap, and heights of most nodes are small.

Analysis of Buid-Max-Heap

EE Analysis

Each call to **MAX-HEAPIFY** costs $O(\lg n)$, and there are $O(n)$ such calls. Thus, the running time is $O(n \cdot \lg n)$.

CS Analysis

- The time for **MAX-HEAPIFY** to run at a node varies with the height of the node in the heap, and heights of most nodes are small.
- In a heap with n nodes, there are $\lfloor \frac{n}{2} \rfloor$ nonleaf nodes, and thus there are $n - \lfloor \frac{n}{2} \rfloor = \lceil \frac{n}{2} \rceil$ nodes with height 0.

Analysis of Buid-Max-Heap

EE Analysis

Each call to **MAX-HEAPIFY** costs $O(\lg n)$, and there are $O(n)$ such calls. Thus, the running time is $O(n \cdot \lg n)$.

CS Analysis

- The time for **MAX-HEAPIFY** to run at a node varies with the height of the node in the heap, and heights of most nodes are small.
- In a heap with n nodes, there are $\lfloor \frac{n}{2} \rfloor$ nonleaf nodes, and thus there are $n - \lfloor \frac{n}{2} \rfloor = \lceil \frac{n}{2} \rceil$ nodes with height 0.
- How many height 1 nodes are there?

Analysis of Buid-Max-Heap

EE Analysis

Each call to **MAX-HEAPIFY** costs $O(\lg n)$, and there are $O(n)$ such calls. Thus, the running time is $O(n \cdot \lg n)$.

CS Analysis

- The time for **MAX-HEAPIFY** to run at a node varies with the height of the node in the heap, and heights of most nodes are small.
- In a heap with n nodes, there are $\lfloor \frac{n}{2} \rfloor$ nonleaf nodes, and thus there are $n - \lfloor \frac{n}{2} \rfloor = \lceil \frac{n}{2} \rceil$ nodes with height 0.
- How many height 1 nodes are there? $\lceil \frac{\lceil \frac{n}{2} \rceil}{2} \rceil = \lceil \frac{n}{2^2} \rceil$.

Analysis of Buid-Max-Heap

EE Analysis

Each call to **MAX-HEAPIFY** costs $O(\lg n)$, and there are $O(n)$ such calls. Thus, the running time is $O(n \cdot \lg n)$.

CS Analysis

- The time for **MAX-HEAPIFY** to run at a node varies with the height of the node in the heap, and heights of most nodes are small.
- In a heap with n nodes, there are $\lfloor \frac{n}{2} \rfloor$ nonleaf nodes, and thus there are $n - \lfloor \frac{n}{2} \rfloor = \lceil \frac{n}{2} \rceil$ nodes with height 0.
- How many height 1 nodes are there? $\lceil \frac{\lceil \frac{n}{2} \rceil}{2} \rceil = \lceil \frac{n}{2^2} \rceil$.
- **An n -element heap has height $\lfloor \lg n \rfloor$ and at most $\lceil \frac{n}{2^{h+1}} \rceil$ nodes of any height h . Moreover, MAX-HEAPIFY when called on a node of height h takes $O(h)$ time.**

Analysis of Buid-Max-Heap

EE Analysis

Each call to **MAX-HEAPIFY** costs $O(\lg n)$, and there are $O(n)$ such calls. Thus, the running time is $O(n \cdot \lg n)$.

CS Analysis

- The time for **MAX-HEAPIFY** to run at a node varies with the height of the node in the heap, and heights of most nodes are small.
- In a heap with n nodes, there are $\lfloor \frac{n}{2} \rfloor$ nonleaf nodes, and thus there are $n - \lfloor \frac{n}{2} \rfloor = \lceil \frac{n}{2} \rceil$ nodes with height 0.
- How many height 1 nodes are there? $\lceil \frac{\lceil \frac{n}{2} \rceil}{2} \rceil = \lceil \frac{n}{2^2} \rceil$.
- **An n -element heap has height $\lfloor \lg n \rfloor$ and at most $\lceil \frac{n}{2^{h+1}} \rceil$ nodes of any height h . Moreover, **MAX-HEAPIFY** when called on a node of height h takes $O(h)$ time.**
- Thus, the time it takes is $O(\sum_{h=0}^{\lfloor \lg n \rfloor} \lceil \frac{n}{2^{h+1}} \rceil \cdot h)$

Analysis of Buid-Max-Heap

EE Analysis

Each call to **MAX-HEAPIFY** costs $O(\lg n)$, and there are $O(n)$ such calls. Thus, the running time is $O(n \cdot \lg n)$.

CS Analysis

- The time for **MAX-HEAPIFY** to run at a node varies with the height of the node in the heap, and heights of most nodes are small.
- In a heap with n nodes, there are $\lfloor \frac{n}{2} \rfloor$ nonleaf nodes, and thus there are $n - \lfloor \frac{n}{2} \rfloor = \lceil \frac{n}{2} \rceil$ nodes with height 0.
- How many height 1 nodes are there? $\lceil \frac{\lceil \frac{n}{2} \rceil}{2} \rceil = \lceil \frac{n}{2^2} \rceil$.
- **An n -element heap has height $\lfloor \lg n \rfloor$ and at most $\lceil \frac{n}{2^{h+1}} \rceil$ nodes of any height h . Moreover, **MAX-HEAPIFY** when called on a node of height h takes $O(h)$ time.**
- Thus, the time it takes is $O(\sum_{h=0}^{\lfloor \lg n \rfloor} \lceil \frac{n}{2^{h+1}} \rceil \cdot h) = O\left(n \cdot \sum_{h=0}^{\lfloor \lg n \rfloor} \frac{h}{2^h}\right)$

Analysis of Buid-Max-Heap

EE Analysis

Each call to **MAX-HEAPIFY** costs $O(\lg n)$, and there are $O(n)$ such calls. Thus, the running time is $O(n \cdot \lg n)$.

CS Analysis

- The time for **MAX-HEAPIFY** to run at a node varies with the height of the node in the heap, and heights of most nodes are small.
- In a heap with n nodes, there are $\lfloor \frac{n}{2} \rfloor$ nonleaf nodes, and thus there are $n - \lfloor \frac{n}{2} \rfloor = \lceil \frac{n}{2} \rceil$ nodes with height 0.
- How many height 1 nodes are there? $\lceil \frac{\lceil \frac{n}{2} \rceil}{2} \rceil = \lceil \frac{n}{2^2} \rceil$.
- **An n -element heap has height $\lfloor \lg n \rfloor$ and at most $\lceil \frac{n}{2^{h+1}} \rceil$ nodes of any height h . Moreover, **MAX-HEAPIFY** when called on a node of height h takes $O(h)$ time.**
- Thus, the time it takes is $O(\sum_{h=0}^{\lfloor \lg n \rfloor} \lceil \frac{n}{2^{h+1}} \rceil \cdot h) = O\left(n \cdot \sum_{h=0}^{\lfloor \lg n \rfloor} \frac{h}{2^h}\right) = O\left(n \cdot \sum_{h=0}^{\infty} (h \cdot (\frac{1}{2})^h)\right)$

Analysis of Buid-Max-Heap

EE Analysis

Each call to **MAX-HEAPIFY** costs $O(\lg n)$, and there are $O(n)$ such calls. Thus, the running time is $O(n \cdot \lg n)$.

CS Analysis

- The time for **MAX-HEAPIFY** to run at a node varies with the height of the node in the heap, and heights of most nodes are small.
- In a heap with n nodes, there are $\lfloor \frac{n}{2} \rfloor$ nonleaf nodes, and thus there are $n - \lfloor \frac{n}{2} \rfloor = \lceil \frac{n}{2} \rceil$ nodes with height 0.
- How many height 1 nodes are there? $\lceil \frac{\lceil \frac{n}{2} \rceil}{2} \rceil = \lceil \frac{n}{2^2} \rceil$.
- **An n -element heap has height $\lfloor \lg n \rfloor$ and at most $\lceil \frac{n}{2^{h+1}} \rceil$ nodes of any height h . Moreover, **MAX-HEAPIFY** when called on a node of height h takes $O(h)$ time.**
- Thus, the time it takes is $O(\sum_{h=0}^{\lfloor \lg n \rfloor} \lceil \frac{n}{2^{h+1}} \rceil \cdot h) = O\left(n \cdot \sum_{h=0}^{\lfloor \lg n \rfloor} \frac{h}{2^h}\right) = O\left(n \cdot \sum_{h=0}^{\infty} (h \cdot (\frac{1}{2})^h)\right) = O(n \cdot 2) = O(n)$.

Analysis of Buid-Max-Heap

EE Analysis

Each call to **MAX-HEAPIFY** costs $O(\lg n)$, and there are $O(n)$ such calls. Thus, the running time is $O(n \cdot \lg n)$.

CS Analysis

- The time for **MAX-HEAPIFY** to run at a node varies with the height of the node in the heap, and heights of most nodes are small.
- In a heap with n nodes, there are $\lfloor \frac{n}{2} \rfloor$ nonleaf nodes, and thus there are $n - \lfloor \frac{n}{2} \rfloor = \lceil \frac{n}{2} \rceil$ nodes with height 0.
- How many height 1 nodes are there? $\lceil \frac{\lceil \frac{n}{2} \rceil}{2} \rceil = \lceil \frac{n}{2^2} \rceil$.
- **An n -element heap has height $\lfloor \lg n \rfloor$ and at most $\lceil \frac{n}{2^{h+1}} \rceil$ nodes of any height h . Moreover, **MAX-HEAPIFY** when called on a node of height h takes $O(h)$ time.**
- Thus, the time it takes is $O(\sum_{h=0}^{\lfloor \lg n \rfloor} \lceil \frac{n}{2^{h+1}} \rceil \cdot h) = O\left(n \cdot \sum_{h=0}^{\lfloor \lg n \rfloor} \frac{h}{2^h}\right) = O\left(n \cdot \sum_{h=0}^{\infty} (h \cdot (\frac{1}{2})^h)\right) = O(n \cdot 2) = O(n)$. Since the algorithm takes $\Omega(n)$ time, the running time is $\Theta(n)$.

Heapsort

Description

Builds a max-heap on the input array $A[1 \dots n]$.

Heapsort

Description

Builds a max-heap on the input array $A[1 \dots n]$. The maximum element ($A[1]$) can be put into its correct position ($A[n]$) by exchanging it with $A[n]$.

Heapsort

Description

Builds a max-heap on the input array $A[1 \dots n]$. The maximum element ($A[1]$) can be put into its correct position ($A[n]$) by exchanging it with $A[n]$. If we now discard node n from the heap (by decrementing **heap-size[A]**) and **MAX-HEAPIFY**, we will have the largest element in $A[n]$, and a max-heap in $A[1 \dots (n - 1)]$.

Heapsort

Description

Builds a max-heap on the input array $A[1 \dots n]$. The maximum element ($A[1]$) can be put into its correct position ($A[n]$) by exchanging it with $A[n]$. If we now discard node n from the heap (by decrementing **heap-size[A]**) and **MAX-HEAPIFY**, we will have the largest element in $A[n]$, and a max-heap in $A[1 \dots (n - 1)]$. Repeating this process will give us a sorting algorithm.

Heapsort

Description

Builds a max-heap on the input array $A[1 \dots n]$. The maximum element ($A[1]$) can be put into its correct position ($A[n]$) by exchanging it with $A[n]$. If we now discard node n from the heap (by decrementing **heap-size[A]**) and **MAX-HEAPIFY**, we will have the largest element in $A[n]$, and a max-heap in $A[1 \dots (n-1)]$. Repeating this process will give us a sorting algorithm.

Pseudocode

```
HEAPSORT (A)
    BUILD-MAX-HEAP (A) ;
    for  $i = \text{length}[A]$  downto 2
        exchange  $A[1] \leftrightarrow A[i]$ ;
        heap-size[A] = heap-size[A] - 1;
        MAX-HEAPIFY (A, 1) ;
```

Heapsort

Description

Builds a max-heap on the input array $A[1 \dots n]$. The maximum element ($A[1]$) can be put into its correct position ($A[n]$) by exchanging it with $A[n]$. If we now discard node n from the heap (by decrementing **heap-size[A]**) and **MAX-HEAPIFY**, we will have the largest element in $A[n]$, and a max-heap in $A[1 \dots (n-1)]$. Repeating this process will give us a sorting algorithm.

Pseudocode

```
HEAPSORT (A)  
  BUILD-MAX-HEAP (A) ;  
  for  $i = \text{length}[A]$  downto 2  
    exchange  $A[1] \leftrightarrow A[i]$ ;  
    heap-size[A] = heap-size[A] - 1;  
    MAX-HEAPIFY (A, 1) ;
```

Loop Invariant and Time Complexity

At the start of each iteration of the **for** loop, the subarray $A[1 \dots i]$ is a max-heap containing the i smallest elements of A , and the subarray $A[(i+1) \dots n]$ contains the $(n-i)$ largest elements of A sorted.

Heapsort

Description

Builds a max-heap on the input array $A[1 \dots n]$. The maximum element ($A[1]$) can be put into its correct position ($A[n]$) by exchanging it with $A[n]$. If we now discard node n from the heap (by decrementing **heap-size[A]**) and **MAX-HEAPIFY**, we will have the largest element in $A[n]$, and a max-heap in $A[1 \dots (n-1)]$. Repeating this process will give us a sorting algorithm.

Pseudocode

```
HEAPSORT (A)  
  BUILD-MAX-HEAP (A) ;  
  for  $i = \text{length}[A]$  downto 2  
    exchange  $A[1] \leftrightarrow A[i]$ ;  
    heap-size[A] = heap-size[A] - 1;  
    MAX-HEAPIFY (A, 1) ;
```

Loop Invariant and Time Complexity

At the start of each iteration of the **for** loop, the subarray $A[1 \dots i]$ is a max-heap containing the i smallest elements of A , and the subarray $A[(i+1) \dots n]$ contains the $(n-i)$ largest elements of A sorted. Algorithm runs in time $O(n \cdot \lg n)$.

Priority Queues

Philosophical Discussions

- Queues are a data structure, where the element dequeued is the first element enqueued.

Priority Queues

Philosophical Discussions

- Queues are a data structure, where the element dequeued is the first element enqueued.
- Thus, they are sometimes called FIFO(first-in-first-out) data structure.

Priority Queues

Philosophical Discussions

- Queues are a data structure, where the element dequeued is the first element enqueued.
- Thus, they are sometimes called FIFO(first-in-first-out) data structure.
- They treat all elements as if they are equal.

Priority Queues

Philosophical Discussions

- Queues are a data structure, where the element dequeued is the first element enqueued.
- Thus, they are sometimes called FIFO(first-in-first-out) data structure.
- They treat all elements as if they are equal.
- “*All animals are equal, but some animals are more equal than others*” (**George Orwell, Animal Farm**, 1945).

Priority Queues

Philosophical Discussions

- Queues are a data structure, where the element dequeued is the first element enqueued.
- Thus, they are sometimes called FIFO(first-in-first-out) data structure.
- They treat all elements as if they are equal.
- “*All animals are equal, but some animals are more equal than others*” (**George Orwell, Animal Farm**, 1945).
- Priority queues are implementing this idea! We will next implement a max-priority queue by using max-heaps.

Max-Priority Queue Operations

List of Operations

- **INSERT(S,x)**: inserts the element x into the set S . This operation could be written as $S = S \cup \{x\}$.

Max-Priority Queue Operations

List of Operations

- **INSERT(S, x):** inserts the element x into the set S . This operation could be written as $S = S \cup \{x\}$.
- **MAXIMUM(S):** returns the element of S with the largest key.

Max-Priority Queue Operations

List of Operations

- **INSERT(S, x):** inserts the element x into the set S . This operation could be written as $S = S \cup \{x\}$.
- **MAXIMUM(S):** returns the element of S with the largest key.
- **EXTRACT-MAX(S):** removes and returns the element of S with the largest key.

Max-Priority Queue Operations

List of Operations

- **INSERT(S, x):** inserts the element x into the set S . This operation could be written as $S = S \cup \{x\}$.
- **MAXIMUM(S):** returns the element of S with the largest key.
- **EXTRACT-MAX(S):** removes and returns the element of S with the largest key.
- **INCREASE-KEY(S, x, k):** increases the value of element x 's key to the new value k , which is assumed to be at least as large as x 's current key.

Max-Priority Queue Operations

List of Operations

- **INSERT(S,x)**: inserts the element x into the set S . This operation could be written as $S = S \cup \{x\}$.
- **MAXIMUM(S)**: returns the element of S with the largest key.
- **EXTRACT-MAX(S)**: removes and returns the element of S with the largest key.
- **INCREASE-KEY(S,x,k)**: increases the value of element x 's key to the new value k , which is assumed to be at least as large as x 's current key.

Pseudocode for Maximum

```
HEAP-MAXIMUM(A)  
    return A[1];
```

Max-Priority Queue Operations

List of Operations

- **INSERT(S, x):** inserts the element x into the set S . This operation could be written as $S = S \cup \{x\}$.
- **MAXIMUM(S):** returns the element of S with the largest key.
- **EXTRACT-MAX(S):** removes and returns the element of S with the largest key.
- **INCREASE-KEY(S, x, k):** increases the value of element x 's key to the new value k , which is assumed to be at least as large as x 's current key.

Pseudocode for Maximum

```
HEAP-MAXIMUM(A)  
    return A[1];
```

Time Complexity

HEAP-MAXIMUM runs in time $\Theta(1)$.

Extract Max

Pseudocode

```
HEAP-EXTRACT-MAX(A)
  if heap-size[A] < 1
    error "heap underflow";

  max = A[1];
  A[1] = A[heap-size[A]];
  heap-size[A] = heap-size[A] - 1;
  MAX-HEAPIFY(A, 1);
  return max;
```

Extract Max

Pseudocode

```
HEAP-EXTRACT-MAX(A)
  if heap-size[A] < 1
    error "heap underflow";

  max = A[1];
  A[1] = A[heap-size[A]];
  heap-size[A] = heap-size[A] - 1;
  MAX-HEAPIFY(A, 1);
  return max;
```

Time Complexity

HEAP-EXTRACT-MAX runs in time $O(\lg n)$.

Increase Key

Pseudocode

```
HEAP-INCREASE-KEY(A, i, key)
    if key < A[i]
        error "new key is smaller than current key";

    A[i] = key;
    while i > 1 and A[PARENT[i]] < A[i]
        exchange A[i]  $\leftrightarrow$  A[PARENT[i]];
        i = PARENT[i];
```

Increase Key

Pseudocode

```
HEAP-INCREASE-KEY(A, i, key)
    if key < A[i]
        error "new key is smaller than current key";

    A[i] = key;
    while i > 1 and A[PARENT[i]] < A[i]
        exchange A[i] ↔ A[PARENT[i]];
        i = PARENT[i];
```

Time Complexity

HEAP-INCREASE-KEY runs in time $O(\lg n)$.

Insert

Pseudocode

```
MAX-HEAP-INSERT(A, key)
    heap-size[A] = heap-size[A] + 1;
    A[heap-size[A]] = -  $\infty$ ;
    HEAP-INCREASE-KEY(A, heap-size[A], key);
```

Insert

Pseudocode

```
MAX-HEAP-INSERT(A, key)
    heap-size[A] = heap-size[A] + 1;
    A[heap-size[A]] = -∞;
    HEAP-INCREASE-KEY(A, heap-size[A], key);
```

Time Complexity

MAX-HEAP-INSERT runs in time $O(\lg n)$.